Busniss Problem

A global e-commerece company operating across multiple regions manages end to end order fulfillment,including shipping and delivery,for products like sporting goods.The company is facing inconsistent delivery performance,where acrual shipping times often deviate from scheduled timelines,leading to late deliveries and unpredictable order profiability.

Desired Outcomes:

The goal is to analyze delivery operations,identify bottlenecks and build a predictive system to reduce delays,optimize shipping decisions and improve overall profitablity and efficiency.

In [19]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import matplotlib.ticker as ticker
import matplotlib.cm as cm
from warnings import filterwarnings
filterwarnings('ignore')
```

In [20]:
```
sns.set_theme(style="whitegrid", palette="deep")

plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.facecolor'] = '#FFFFFF'
plt.rcParams['axes.facecolor'] = '#F8FAFC'
plt.rcParams['axes.titlecolor'] = '#12304A'
plt.rcParams['axes.labelcolor'] = '#1E293B'
plt.rcParams['grid.color'] = '#E2E8F0'
plt.rcParams['font.family'] = 'DejaVu Sans'
# Professional Color Theme
primary_color = '#1F4E78'      # Corporate Blue
secondary_color = '#5B9BD5'    # Soft Blue
accent_color = '#00A6A6'       # Professional Teal
background_color = '#F4F7FA'   # Light Grey
text_color = '#263238'         # Dark Grey
```

In [21]:
```
df=pd.read_csv('C:/Users/khush/OneDrive/Desktop/Data analysis/Dataset/DataCoSupp
df.head()
```

Out[21]:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Delivery Status	Late_delivery_ris
0	DEBIT	3	4	91.250000	314.640015	Advance shipping	
1	TRANSFER	5	4	-249.089996	311.359985	Late delivery	
2	CASH	4	4	-247.779999	309.720001	Shipping on time	
3	DEBIT	3	4	22.860001	304.809998	Advance shipping	
4	PAYMENT	2	4	134.210007	298.250000	Advance shipping	

5 rows × 53 columns

In [22]:
```
df
```

Out[22]:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Delivery Status	Late_deliv
0	DEBIT	3	4	91.250000	314.640015	Advance shipping	
1	TRANSFER	5	4	-249.089996	311.359985	Late delivery	
2	CASH	4	4	-247.779999	309.720001	Shipping on time	
3	DEBIT	3	4	22.860001	304.809998	Advance shipping	
4	PAYMENT	2	4	134.210007	298.250000	Advance shipping	
...	...	...	...	...	...	...	
180514	CASH	4	4	40.000000	399.980011	Shipping on time	
180515	DEBIT	3	2	-613.770019	395.980011	Late delivery	
180516	TRANSFER	5	4	141.110001	391.980011	Late delivery	
180517	PAYMENT	3	4	186.229996	387.980011	Advance shipping	
180518	PAYMENT	4	4	168.949997	383.980011	Shipping on time	

180519 rows × 53 columns

In [23]:
```
print('rows,cols:',df.shape)
print('\ncolumns:')
print(df.columns.tolist())
print('\nNum duplicates:',df.duplicated().sum())
print('\nMissing values(top 20):')
print(df.isna().sum().sort_values(ascending=False).head(20))
```

```
rows,cols: (180519, 53)

columns:
['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)', 'Benefit pe
r order', 'Sales per customer', 'Delivery Status', 'Late_delivery_risk', 'Categor
y Id', 'Category Name', 'Customer City', 'Customer Country', 'Customer Email', 'C
ustomer Fname', 'Customer Id', 'Customer Lname', 'Customer Password', 'Customer S
egment', 'Customer State', 'Customer Street', 'Customer Zipcode', 'Department I
d', 'Department Name', 'Latitude', 'Longitude', 'Market', 'Order City', 'Order Co
untry', 'Order Customer Id', 'order date (DateOrders)', 'Order Id', 'Order Item C
ardprod Id', 'Order Item Discount', 'Order Item Discount Rate', 'Order Item Id',
'Order Item Product Price', 'Order Item Profit Ratio', 'Order Item Quantity', 'Sa
les', 'Order Item Total', 'Order Profit Per Order', 'Order Region', 'Order Stat
e', 'Order Status', 'Order Zipcode', 'Product Card Id', 'Product Category Id', 'P
roduct Description', 'Product Image', 'Product Name', 'Product Price', 'Product S
tatus', 'shipping date (DateOrders)', 'Shipping Mode']

Num duplicates: 0

Missing values(top 20):
Product Description             180519
Order Zipcode                   155679
Customer Lname                       8
Customer Zipcode                     3
Days for shipment (scheduled)        0
Sales per customer                   0
Benefit per order                    0
Delivery Status                      0
Late_delivery_risk                   0
Customer City                        0
Customer Country                     0
Category Id                          0
Category Name                        0
Customer Fname                       0
Customer Email                       0
Customer Password                    0
Customer Id                          0
Customer Segment                     0
Customer State                       0
Days for shipping (real)             0
dtype: int64
```

In [25]:
```
import pandas as pd

df = pd.read_csv(
    'C:/Users/khush/OneDrive/Desktop/Data analysis/Dataset/DataCoSupplyChainData
    encoding='latin1'
)

print("Original shape:", df.shape)
```

```
Original shape: (180519, 53)
```

In [26]:
```
print(df.columns.tolist())
```

```
['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)', 'Benefit pe
r order', 'Sales per customer', 'Delivery Status', 'Late_delivery_risk', 'Categor
y Id', 'Category Name', 'Customer City', 'Customer Country', 'Customer Email', 'C
ustomer Fname', 'Customer Id', 'Customer Lname', 'Customer Password', 'Customer S
egment', 'Customer State', 'Customer Street', 'Customer Zipcode', 'Department I
d', 'Department Name', 'Latitude', 'Longitude', 'Market', 'Order City', 'Order Co
untry', 'Order Customer Id', 'order date (DateOrders)', 'Order Id', 'Order Item C
ardprod Id', 'Order Item Discount', 'Order Item Discount Rate', 'Order Item Id',
'Order Item Product Price', 'Order Item Profit Ratio', 'Order Item Quantity', 'Sa
les', 'Order Item Total', 'Order Profit Per Order', 'Order Region', 'Order Stat
e', 'Order Status', 'Order Zipcode', 'Product Card Id', 'Product Category Id', 'P
roduct Description', 'Product Image', 'Product Name', 'Product Price', 'Product S
tatus', 'shipping date (DateOrders)', 'Shipping Mode']
```

In [27]:
```python
import pandas as pd

df = pd.read_csv(
    'C:/Users/khush/OneDrive/Desktop/Data analysis/Dataset/DataCoSupplyChainData
    encoding='latin1'
)

print(df['Product Status'].value_counts())
```

```
Product Status
0    180519
Name: count, dtype: int64
```

In [28]:
```python
# Select exactly 20 useful columns
columns_to_keep = [
    'Type',
    'Days for shipping (real)',
    'Days for shipment (scheduled)',
    'Benefit per order',
    'Sales per customer',
    'Delivery Status',
    'Late_delivery_risk',
    'Category Name',
    'Customer Country',
    'Customer Segment',
    'Department Name',
    'Order Item Profit Ratio',
    'Sales',
    'Order Item Total',
    'Order Profit Per Order',
    'Order Region',
    'Order Status',
    'Product Name',
    'Shipping Mode',
    'order date (DateOrders)'
]

# Check whether columns exist
missing_columns = [col for col in columns_to_keep if col not in df.columns]

if missing_columns:
    print("Columns not found:", missing_columns)
else:
    # Keep only the selected columns
    df = df[columns_to_keep].copy()
```

```
# Remove canceled orders
df = df[df['Delivery Status'] != 'Shipping canceled'].copy()

# Convert order date
df['order date (DateOrders)'] = pd.to_datetime(
    df['order date (DateOrders)'],
    errors='coerce'
)

# Display results
print("Rows and columns:", df.shape)
print("\nTotal columns:", len(df.columns))
print("\nColumn names:")
print(df.columns.tolist())

print("\nMissing values (top 5):")
print(df.isna().sum().sort_values(ascending=False).head(5))

display(df.head())
```

```
Rows and columns: (172765, 20)

Total columns: 20

Column names:
['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)', 'Benefit pe
r order', 'Sales per customer', 'Delivery Status', 'Late_delivery_risk', 'Categor
y Name', 'Customer Country', 'Customer Segment', 'Department Name', 'Order Item P
rofit Ratio', 'Sales', 'Order Item Total', 'Order Profit Per Order', 'Order Regio
n', 'Order Status', 'Product Name', 'Shipping Mode', 'order date (DateOrders)']

Missing values (top 5):
Type                            0
Days for shipping (real)        0
Days for shipment (scheduled)   0
Benefit per order               0
Sales per customer              0
dtype: int64
```

	Type	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Delivery Status	Late_delivery_risk
0	DEBIT	3	4	91.250000	314.640015	Advance shipping	0
1	TRANSFER	5	4	-249.089996	311.359985	Late delivery	1
2	CASH	4	4	-247.779999	309.720001	Shipping on time	0
3	DEBIT	3	4	22.860001	304.809998	Advance shipping	0
4	PAYMENT	2	4	134.210007	298.250000	Advance shipping	0

In [29]:
```
df.head()
```

Out[29]:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Delivery Status	Late_delivery_ris
0	DEBIT	3	4	91.250000	314.640015	Advance shipping	
1	TRANSFER	5	4	-249.089996	311.359985	Late delivery	
2	CASH	4	4	-247.779999	309.720001	Shipping on time	
3	DEBIT	3	4	22.860001	304.809998	Advance shipping	
4	PAYMENT	2	4	134.210007	298.250000	Advance shipping	

In [30]:
```
for col in df.columns:
    if df[col].nunique()<10:
        print(f'\n{col},value counts:')
        print(df[col].value_counts())
```

```
Type,value counts:
Type
DEBIT       69295
TRANSFER    42129
PAYMENT     41725
CASH        19616
Name: count, dtype: int64

Days for shipping (real),value counts:
Days for shipping (real)
2    54205
6    27489
3    27478
4    27297
5    27003
0     4839
1     4454
Name: count, dtype: int64

Days for shipment (scheduled),value counts:
Days for shipment (scheduled)
4    103153
2     33806
1     26513
0      9293
Name: count, dtype: int64

Delivery Status,value counts:
Delivery Status
Late delivery       98977
Advance shipping    41592
Shipping on time    32196
Name: count, dtype: int64

Late_delivery_risk,value counts:
Late_delivery_risk
1    98977
0    73788
Name: count, dtype: int64

Customer Country,value counts:
Customer Country
EE. UU.        106425
Puerto Rico     66340
Name: count, dtype: int64

Customer Segment,value counts:
Customer Segment
Consumer       89420
Corporate      52528
Home Office    30817
Name: count, dtype: int64

Order Status,value counts:
Order Status
COMPLETE           59491
PENDING_PAYMENT    39832
PROCESSING         21902
PENDING            20227
CLOSED             19616
```

```
ON_HOLD             9804
PAYMENT_REVIEW      1893
Name: count, dtype: int64

Shipping Mode,value counts:
Shipping Mode
Standard Class    103153
Second Class       33806
First Class        26513
Same Day            9293
Name: count, dtype: int64
```

In [31]:
```
print(df.columns.tolist())
```

```
['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)', 'Benefit per order', 'Sales per customer', 'Delivery Status', 'Late_delivery_risk', 'Category Name', 'Customer Country', 'Customer Segment', 'Department Name', 'Order Item Profit Ratio', 'Sales', 'Order Item Total', 'Order Profit Per Order', 'Order Region', 'Order Status', 'Product Name', 'Shipping Mode', 'order date (DateOrders)']
```

In [32]:
```
print([col for col in df.columns if 'date' in col.lower()])
```

```
['order date (DateOrders)']
```

In [33]:
```
import pandas as pd

df = pd.read_csv(
    'C:/Users/khush/OneDrive/Desktop/Data analysis/Dataset/DataCoSupplyChainData
    encoding='latin1'
)


# Remove extra spaces from column names
df.columns = df.columns.str.strip()

# Convert dates to datetime
df['shipping date (DateOrders)'] = pd.to_datetime(
    df['shipping date (DateOrders)'], errors='coerce'
)

df['order date (DateOrders)'] = pd.to_datetime(
    df['order date (DateOrders)'], errors='coerce'
)

# Calculate processing time and delay
df['Order Processing Time'] = (
    df['shipping date (DateOrders)'] -df['order date (DateOrders)']
).dt.days

df['Delay'] = (
    df['Order Processing Time'] -
    df['Days for shipment (scheduled)']
)

df['Is_Delayed'] = df['Delay'] > 0

# Extract date information
df['order_month'] = df['order date (DateOrders)'].dt.month
df['order_day'] = df['order date (DateOrders)'].dt.day_name()
df['order_hour'] = df['order date (DateOrders)'].dt.hour
```

```
df.describe()
```

Out[33]:

	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Late_delivery_risk
count	180519.000000	180519.000000	180519.000000	180519.000000	180519.000000
mean	3.497654	2.931847	21.974989	183.107609	0.548291
min	0.000000	0.000000	-4274.979980	7.490000	0.000000
25%	2.000000	2.000000	7.000000	104.379997	0.000000
50%	3.000000	4.000000	31.520000	163.990005	1.000000
75%	5.000000	4.000000	64.800003	247.399994	1.000000
max	6.000000	4.000000	911.799988	1939.989990	1.000000
std	1.623722	1.374449	104.433526	120.043670	0.497664

8 rows × 35 columns

In [34]:
```
df['Is_Delayed'].value_counts()
```

Out[34]:
```
Is_Delayed
True     98743
False    81776
Name: count, dtype: int64
```

In [35]:
```
df.columns
```

Out[35]:
```
Index(['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)',
       'Benefit per order', 'Sales per customer', 'Delivery Status',
       'Late_delivery_risk', 'Category Id', 'Category Name', 'Customer City',
       'Customer Country', 'Customer Email', 'Customer Fname', 'Customer Id',
       'Customer Lname', 'Customer Password', 'Customer Segment',
       'Customer State', 'Customer Street', 'Customer Zipcode',
       'Department Id', 'Department Name', 'Latitude', 'Longitude', 'Market',
       'Order City', 'Order Country', 'Order Customer Id',
       'order date (DateOrders)', 'Order Id', 'Order Item Cardprod Id',
       'Order Item Discount', 'Order Item Discount Rate', 'Order Item Id',
       'Order Item Product Price', 'Order Item Profit Ratio',
       'Order Item Quantity', 'Sales', 'Order Item Total',
       'Order Profit Per Order', 'Order Region', 'Order State', 'Order Status',
       'Order Zipcode', 'Product Card Id', 'Product Category Id',
       'Product Description', 'Product Image', 'Product Name', 'Product Price',
       'Product Status', 'shipping date (DateOrders)', 'Shipping Mode',
       'Order Processing Time', 'Delay', 'Is_Delayed', 'order_month',
       'order_day', 'order_hour'],
      dtype='object')
```

In [36]:
```
df['Order Profit Per Order']>0
```

Out[36]:
```
0          True
1         False
2         False
3          True
4          True
          ...
180514     True
180515    False
180516     True
180517     True
180518     True
Name: Order Profit Per Order, Length: 180519, dtype: bool
```

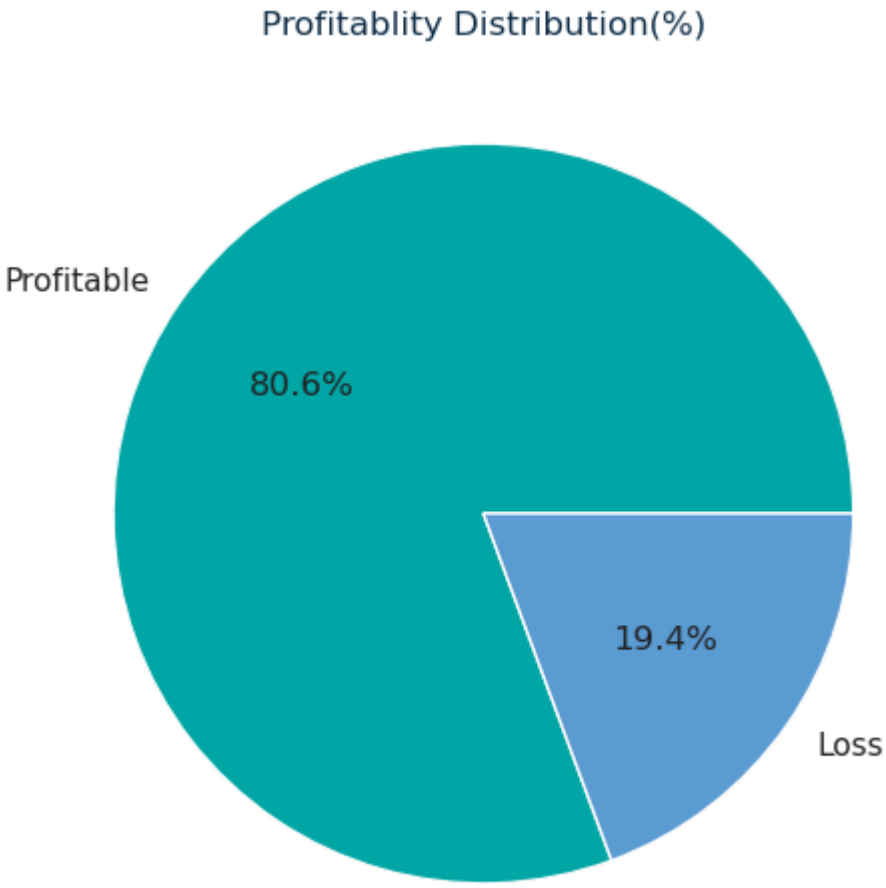
In [37]:
```
# Create profitability flag

df['Profitability_Flag'] = df['Order Profit Per Order'].apply(
    lambda x: 'Profitable' if x > 0 else 'Loss'
)


df['Profitability_Flag'].value_counts()
```

Out[37]:
```
Profitability_Flag
Profitable    145558
Loss           34961
Name: count, dtype: int64
```

In [38]:
```
#visualization of profitability distribution
profit_counts=df['Profitability_Flag'].value_counts(normalize=True)*100
profit_counts.plot(kind='pie',autopct='%1.1f%%',colors=[accent_color,secondary_c
plt.ylabel('')
plt.title('Profitablity Distribution(%)')
plt.show()
```



In [39]:
```
# Business KPI

def format_func(value):
    if value>=1e6:
        return f'{value/1e6:1f}M $'
    elif value>=1e3:
        return f'{value/1e3:1f}K $'
    else:
        return f'{value:0f} $'

delayed_df=df[df['Delay']>0]
metrics={}
metrics['Total Orders']=len(df)
metrics['Late Delivers']=len(delayed_df)
metrics['90% Delay(days)']=delayed_df['Delay'].quantile(0.90)
metrics['On time Delivery %']=(1-float(metrics['Late Delivers'])/ metrics['Total
metrics['Late Delivery %']=float(metrics['Late Delivers'])/ metrics['Total Order
metrics['Total Profit']=format_func(df.loc[df['Order Profit Per Order']>0,'Order
metrics['Total Loss due to delays']=format_func(df.loc[df['Delay']>0,'Order Prof

print('\n---Business KPIs ---\n')
for k,v in metrics.items():
    if isinstance(v,float):
        print(f"{k}:{v:2f}")
    else:
        print(f"{k}:{v}")
```

```
---Business KPIs ---

Total Orders:180519
Late Delivers:98743
90% Delay(days):3.000000
On time Delivery %:45.300495
Late Delivery %:54.699505
Total Profit:7.850450M $
Total Loss due to delays:2.145747M $
```

In [40]:
```
# Profitability vs Delivery Time Analysis

profit_metrics=(
    df.groupby('Delay')['Order Profit Per Order']
    .agg(
        mean_profit='mean',
        total_profit='sum',
        order_count='count'
    )
    .reset_index()
)
```

In [41]:
```
profit_metrics
```

Out[41]:

	Delay	mean_profit	total_profit	order_count
0	-2	23.423528	5.074942e+05	21666
1	-1	21.516997	4.669188e+05	21700
2	0	22.044854	8.467429e+05	38410
3	1	22.225592	1.244411e+06	55990
4	2	21.246239	6.101495e+05	28718
5	3	19.941130	1.406248e+05	7052
6	4	21.561202	1.505619e+05	6983

In [42]:
```
delay_distribution=(
    df['Delay']
    .value_counts(normalize=True)
    .sort_index()*100
).reset_index()
```

In [43]:
```
delay_distribution
```

Out[43]:

	Delay	proportion
0	-2	12.002061
1	-1	12.020895
2	0	21.277539
3	1	31.016126
4	2	15.908575
5	3	3.906514
6	4	3.868291

In [44]:

```
primary_color = '#264653'  # Professional dark blue
accent_color = '#2A9D8F'   # Professional teal


delay_distribution.columns = ['Delay_Days', 'Percentage']

print("\nProfit metrics by delay day:")
display(profit_metrics.round(1))

print("\nDelay Distribution (%):")
display(delay_distribution)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# First subplot: Delay Distribution
sns.barplot(
    x='Delay_Days',
    y='Percentage',
    data=delay_distribution,
    color=accent_color,
    ax=ax1
)

ax1.set_title('Delay Distribution')
ax1.set_xlabel('Delay (Days)')
ax1.set_ylabel('Percentage of Orders (%)')

# Percentage text on bars
for bar in ax1.patches:
    height = bar.get_height()
    ax1.text(
        bar.get_x() + bar.get_width() / 2,
        height + 0.5,
        f'{height:.1f}%',
        ha='center',
        va='bottom'
    )

# Second subplot: Profit Analysis by Delay Days
ax2.set_title('Profit Analysis by Delay Days')

ax2.bar(
    profit_metrics['Delay'],
    profit_metrics['total_profit'],
```

```
    color=primary_color,
    label='Total Profit'
)

ax2.set_xlabel('Delay Days')
ax2.set_ylabel('Total Profit', color=primary_color)
ax2.tick_params(axis='y', labelcolor=primary_color)

# Secondary Y-axis
ax3 = ax2.twinx()

ax3.set_ylabel('Mean Profit', color=accent_color)

ax3.plot(
    profit_metrics['Delay'],
    profit_metrics['mean_profit'],
    marker='o',
    label='Mean Profit',
    color=accent_color
)

ax3.tick_params(axis='y', labelcolor=accent_color)

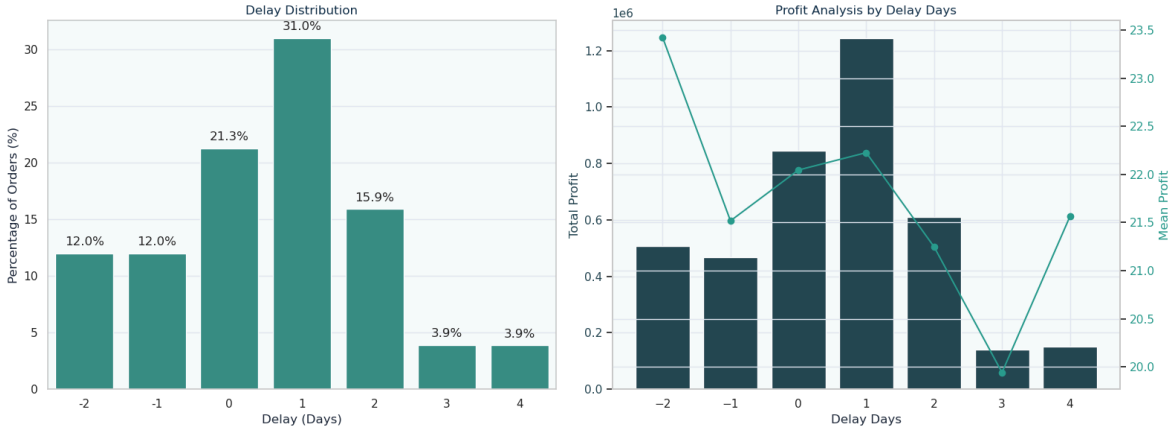
plt.tight_layout()
plt.show()
```

```
Profit metrics by delay day:
```

	Delay	mean_profit	total_profit	order_count
0	-2	23.4	507494.2	21666
1	-1	21.5	466918.8	21700
2	0	22.0	846742.9	38410
3	1	22.2	1244410.9	55990
4	2	21.2	610149.5	28718
5	3	19.9	140624.8	7052
6	4	21.6	150561.9	6983

```
Delay Distribution (%):
```

	Delay_Days	Percentage
0	-2	12.002061
1	-1	12.020895
2	0	21.277539
3	1	31.016126
4	2	15.908575
5	3	3.906514
6	4	3.868291



In [45]:
```
# Bottleneck Detection

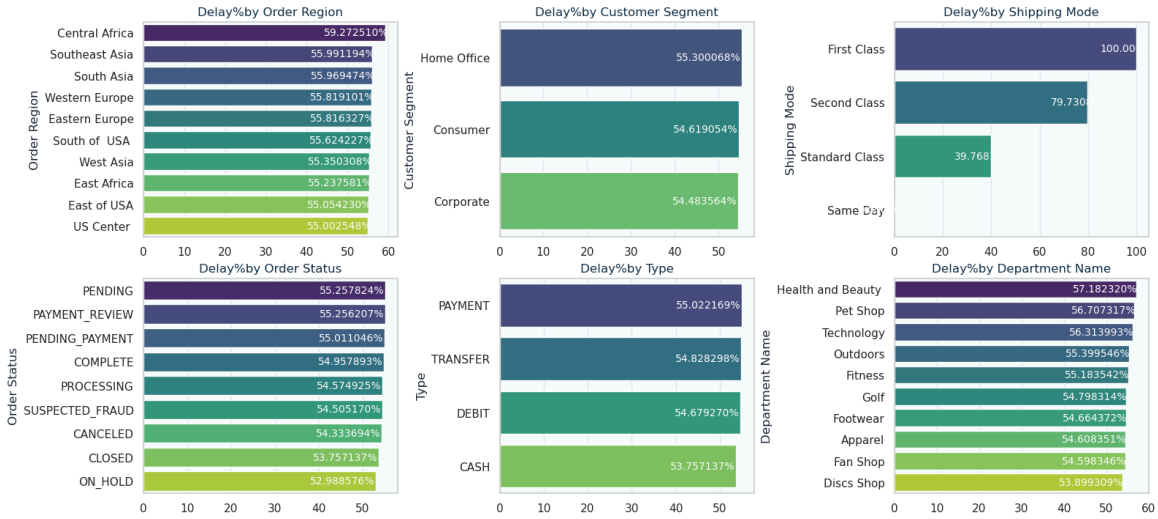
def compute_delay_pct_by_category(category):
    cat_df=df.groupby(category).agg(
        total_orders=('Delay','count'),
        late_orders=('Is_Delayed','sum')
    ).reset_index()
    cat_df['delay_pct']=cat_df['late_orders']/cat_df['total_orders']*100
    cat_df=cat_df.sort_values('delay_pct',ascending=False).head(10)
    return cat_df

categories=['Order Region','Customer Segment','Shipping Mode','Order Status','Ty

fig,axes=plt.subplots(2,3,figsize=(16,7),constrained_layout=True)
axes=axes.flatten()

for ax,category in zip(axes,categories):
    cat_df=compute_delay_pct_by_category(category)
    sns.barplot(
        data=cat_df,
        x='delay_pct',
        y=category,
        ax=ax,
        palette='viridis'
    )
    ax.set_title(f'Delay%by {category}')
    ax.set_xlabel('')
    ax.set_ylabel(category)
    for i,row in cat_df.reset_index().iterrows():
            ax.text(row['delay_pct']-15,i,f"{row['delay_pct']:1f}%",va='center',

plt.show()
```



In [46]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(
    'C:/Users/khush/OneDrive/Desktop/Data analysis/Dataset/DataCoSupplyChainData
    encoding='latin1'
)



# Create Delay column if it does not exist
if 'Delay' not in df.columns:
    df['Delay'] = (
        df['Days for shipping (real)']
        - df['Days for shipment (scheduled)']
    )

# Create Is_Delayed column if it does not exist
if 'Is_Delayed' not in df.columns:
    df['Is_Delayed'] = (df['Delay'] > 0).astype(int)


# Root Cause Analysis
# Top drivers of late delivery by region

def top_drivers_for_region(region):

    df_region = df[df['Order Region'] == region].copy()

    if df_region.empty:
        print(f"No data found for region: {region}")
        print("Available regions:")
        print(df['Order Region'].dropna().unique())
        return

    drivers = [
        'Shipping Mode',
        'Customer Segment',
        'Department Name',
        'Type',
        'Order Status'
    ]
```

```
all_factors = []

for factor in drivers:

    temp = (
        df_region.groupby(factor, dropna=False)
        .agg(
            total_orders=('Delay', 'count'),
            late_orders=('Is_Delayed', 'sum'),
            avg_delay=('Delay', 'mean')
        )
        .reset_index()
    )

    temp['delay_pct'] = (
        temp['late_orders']
        / temp['total_orders']
        * 100
    )

    temp['Driver'] = factor

    temp['Factor_Level'] = (
        factor + ': ' + temp[factor].astype(str)
    )

    all_factors.append(
        temp[
            [
                'Driver',
                'Factor_Level',
                'delay_pct',
                'avg_delay',
                'total_orders'
            ]
        ]
    )

# Combine all drivers
final_df = pd.concat(all_factors, ignore_index=True)

# Select top 10 drivers
top_factors = (
    final_df.sort_values(
        'delay_pct',
        ascending=False
    )
    .head(10)
    .sort_values('delay_pct')
)

# Professional visualization
fig, ax = plt.subplots(figsize=(12, 7))

bars = ax.barh(
    top_factors['Factor_Level'],
    top_factors['delay_pct'],
    color=primary_color
)
```

```
    ax.set_xlabel('Late Delivery Percentage (%)')
    ax.set_ylabel('Driver Factors')

    ax.set_title(
        f'Top Drivers of Late Delivery in {region}',
        fontsize=14,
        fontweight='bold'
    )

    ax.grid(axis='x', linestyle=':', alpha=0.4)
    ax.set_axisbelow(True)

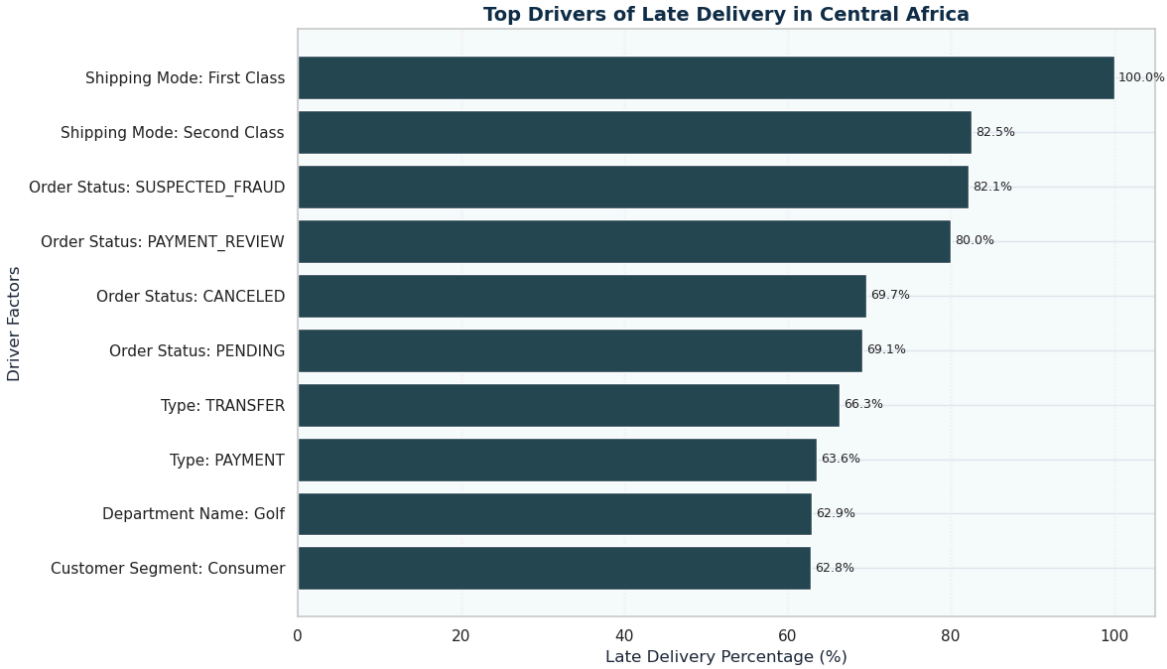
    # Percentage labels
    for bar in bars:
        width = bar.get_width()

        ax.text(
            width + 0.5,
            bar.get_y() + bar.get_height() / 2,
            f'{width:.1f}%',
            va='center',
            fontsize=9
        )

    plt.tight_layout()
    plt.show()

    return top_factors


# Run analysis
top_drivers_for_region('Central Africa')
```



Out[46]:

	Driver	Factor_Level	delay_pct	avg_delay	total_orders
4	Customer Segment	Customer Segment: Consumer	62.787879	0.683636	825
11	Department Name	Department Name: Golf	62.931034	0.698276	348
15	Type	Type: PAYMENT	63.583815	0.763006	346
16	Type	Type: TRANSFER	66.313559	0.667373	472
22	Order Status	Order Status: PENDING	69.148936	0.574468	188
17	Order Status	Order Status: CANCELED	69.696970	0.878788	33
21	Order Status	Order Status: PAYMENT_REVIEW	80.000000	1.200000	20
25	Order Status	Order Status: SUSPECTED_FRAUD	82.142857	1.035714	28
2	Shipping Mode	Shipping Mode: Second Class	82.491582	2.127946	297
0	Shipping Mode	Shipping Mode: First Class	100.000000	1.000000	300

In [47]:
```
print(df.columns.tolist())
```

```
['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)', 'Benefit pe
r order', 'Sales per customer', 'Delivery Status', 'Late_delivery_risk', 'Categor
y Id', 'Category Name', 'Customer City', 'Customer Country', 'Customer Email', 'C
ustomer Fname', 'Customer Id', 'Customer Lname', 'Customer Password', 'Customer S
egment', 'Customer State', 'Customer Street', 'Customer Zipcode', 'Department I
d', 'Department Name', 'Latitude', 'Longitude', 'Market', 'Order City', 'Order Co
untry', 'Order Customer Id', 'order date (DateOrders)', 'Order Id', 'Order Item C
ardprod Id', 'Order Item Discount', 'Order Item Discount Rate', 'Order Item Id',
'Order Item Product Price', 'Order Item Profit Ratio', 'Order Item Quantity', 'Sa
les', 'Order Item Total', 'Order Profit Per Order', 'Order Region', 'Order Stat
e', 'Order Status', 'Order Zipcode', 'Product Card Id', 'Product Category Id', 'P
roduct Description', 'Product Image', 'Product Name', 'Product Price', 'Product S
tatus', 'shipping date (DateOrders)', 'Shipping Mode', 'Delay', 'Is_Delayed']
```

In [50]:
```
# Convert order date to datetime
df['order date (DateOrders)'] = pd.to_datetime(
    df['order date (DateOrders)'], errors='coerce'
)

# Extract time-based columns
df['order_month'] = df['order date (DateOrders)'].dt.month
df['order_day'] = df['order date (DateOrders)'].dt.dayofweek
df['order_hour'] = df['order date (DateOrders)'].dt.hour
```

In [52]:
```
# Delay percentage by month
delay_by_month = (
    df.groupby('order_month')['Is_Delayed']
    .mean()
    .reset_index()
)
delay_by_month['delay_pct'] = delay_by_month['Is_Delayed'] * 100
```

```
# Delay percentage by day
delay_by_day = (
    df.groupby('order_day')['Is_Delayed']
    .mean()
    .reset_index()
)
delay_by_day['delay_pct'] = delay_by_day['Is_Delayed'] * 100

# Delay percentage by hour
delay_by_hour = (
    df.groupby('order_hour')['Is_Delayed']
    .mean()
    .reset_index()
)
delay_by_hour['delay_pct'] = delay_by_hour['Is_Delayed'] * 100
```

In [53]:
```
delay_by_month
```

Out[53]:

	order_month	Is_Delayed	delay_pct
0	1	0.568663	56.866344
1	2	0.572579	57.257898
2	3	0.575853	57.585275
3	4	0.567153	56.715258
4	5	0.576302	57.630195
5	6	0.570579	57.057930
6	7	0.570280	57.028012
7	8	0.578997	57.899698
8	9	0.576603	57.660275
9	10	0.568506	56.850637
10	11	0.568480	56.848000
11	12	0.579442	57.944218

In [55]:
```
import matplotlib.pyplot as plt
import pandas as pd

# Define professional chart colors
primary_color = '#4472C4'  # Professional blue
danger_color = '#C00000'   # Dark red for highlighting
fig,(ax1,ax2,ax3)=plt.subplots(1,3,figsize=(18,6))

# Subplot 1:Delay % Trend over month
ax1.plot(delay_by_month['order_month'],delay_by_month['delay_pct'],marker='o',co
ax1.set_xticks(range(1,13))
ax1.set_xticklabels(['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct'
ax1.set_xlabel("Month")
ax1.set_ylabel("Delay Percentage(%)")
ax1.set_title("Delay % Trend Over Month")
ax1.grid(True,linestyle=':',alpha=0.5)
```

```
# Annotate top 3 highest
top3_month=delay_by_month.nlargest(3,'delay_pct')
for _, row in top3_month.iterrows():
    ax1.annotate(f"{row['delay_pct']:.1f}%",(row['order_month'],row['delay_pct']
                 textcoords="offset points", xytext=(0,10), ha='center', fontsiz


# Subplot 2: Delay % by Day of week
day_order=['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday
delay_by_day['order_day']=pd.Categorical(delay_by_day['order_day'],categories=da
delay_by_day=delay_by_day.sort_values('order_day')

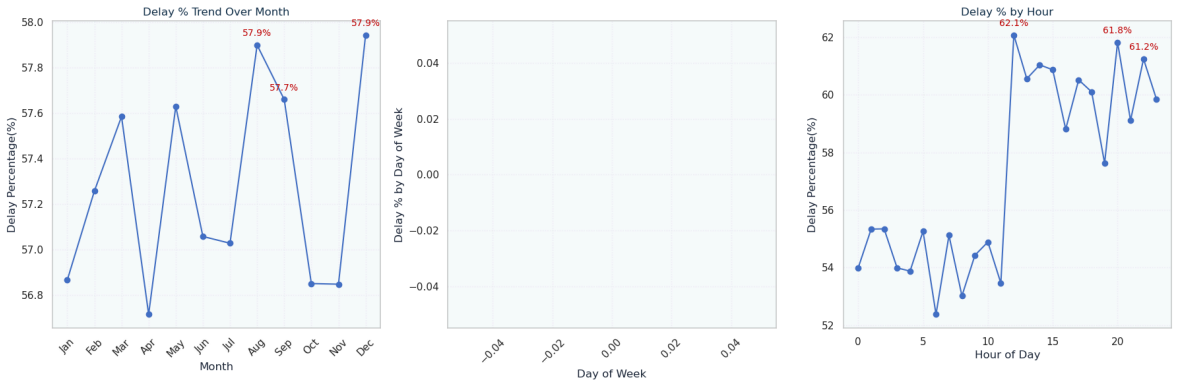
ax2.bar(delay_by_day['order_day'],delay_by_day['delay_pct'],color=primary_color)
ax2.tick_params(axis='x',rotation=45)
ax2.set_xlabel("Day of Week")
ax2.set_ylabel("Delay % by Day of Week")
ax2.grid(True,linestyle=':',alpha=0.5)

#Annotate top 3 highest bars
top3_day=delay_by_day.nlargest(3,'delay_pct')
for _,row in top3_day.iterrows():
     height=row['delay_pct']
     ax2.text(row['order_day'],height+0.5,f'{height:.1f}%', ha='center', fontsiz

# Subplot 3: delay percentage by hour
ax3.plot(delay_by_hour['order_hour'],delay_by_hour['delay_pct'],marker='o',color
ax3.set_xlabel("Hour of Day")
ax3.set_ylabel("Delay Percentage(%)")
ax3.set_title("Delay % by Hour")
ax3.grid(True,linestyle=':',alpha=0.5)


#Annotate top 3 highest bars
top3_hour=delay_by_hour.nlargest(3,'delay_pct')
for _,row in top3_hour.iterrows():
     ax3.annotate(f"{row['delay_pct']:.1f}%",(row['order_hour'],row['delay_pct']
                 textcoords="offset points", xytext=(0,10), ha='center', fontsiz


plt.tight_layout()
plt.show()
```



In [59]:
```
# Convert order date to datetime
df['order date (DateOrders)'] = pd.to_datetime(
    df['order date (DateOrders)'],
    errors='coerce'
)
```

```
# Extract day of week
df['order_day'] = df['order date (DateOrders)'].dt.day_name()
```

In [60]:
```
delay_by_day = (
    df.groupby('order_day')['Is_Delayed']
    .mean()
    .mul(100)
    .reset_index(name='delay_pct')
)
```

In [63]:
```
day_order = [
    'Monday', 'Tuesday', 'Wednesday', 'Thursday',
    'Friday', 'Saturday', 'Sunday'
]

delay_by_day['order_day'] = pd.Categorical(
    delay_by_day['order_day'],
    categories=day_order,
    ordered=True
)

delay_by_day = delay_by_day.sort_values('order_day')

print(delay_by_day)
```

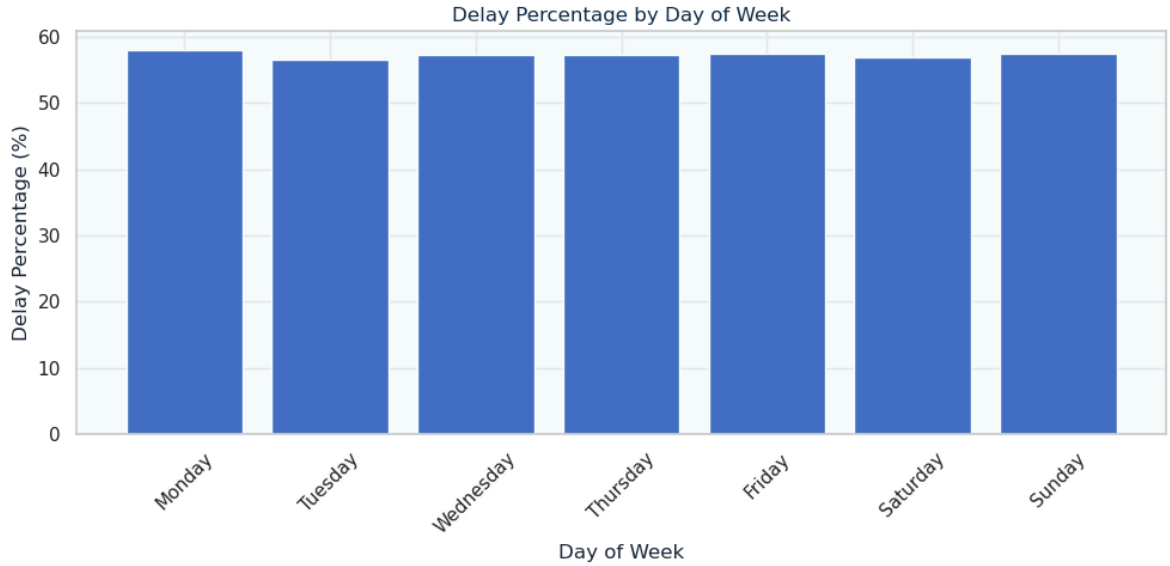
```
   order_day  delay_pct
1     Monday  57.930660
5    Tuesday  56.603700
6  Wednesday  57.189198
4   Thursday  57.238273
0     Friday  57.527483
2   Saturday  56.939886
3     Sunday  57.519464
```

In [66]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

plt.bar(
    delay_by_day['order_day'].astype(str),
    delay_by_day['delay_pct'],
    color='#4472C4'
)

plt.xlabel("Day of Week")
plt.ylabel("Delay Percentage (%)")
plt.title("Delay Percentage by Day of Week")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [68]:
```
# Machine Learning Modeling

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,precision_score,recall_score,classifi
from sklearn.ensemble import RandomForestClassifier
from collections import Counter
from imblearn.over_sampling import SMOTE
```

In [71]:
```
print(df.columns.tolist())
```

```
['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)', 'Benefit pe
r order', 'Sales per customer', 'Delivery Status', 'Late_delivery_risk', 'Categor
y Id', 'Category Name', 'Customer City', 'Customer Country', 'Customer Email', 'C
ustomer Fname', 'Customer Id', 'Customer Lname', 'Customer Password', 'Customer S
egment', 'Customer State', 'Customer Street', 'Customer Zipcode', 'Department I
d', 'Department Name', 'Latitude', 'Longitude', 'Market', 'Order City', 'Order Co
untry', 'Order Customer Id', 'order date (DateOrders)', 'Order Id', 'Order Item C
ardprod Id', 'Order Item Discount', 'Order Item Discount Rate', 'Order Item Id',
'Order Item Product Price', 'Order Item Profit Ratio', 'Order Item Quantity', 'Sa
les', 'Order Item Total', 'Order Profit Per Order', 'Order Region', 'Order Stat
e', 'Order Status', 'Order Zipcode', 'Product Card Id', 'Product Category Id', 'P
roduct Description', 'Product Image', 'Product Name', 'Product Price', 'Product S
tatus', 'shipping date (DateOrders)', 'Shipping Mode', 'Delay', 'Is_Delayed', 'or
der_month', 'order_day', 'order_hour']
```

In [72]:
```
# Independent variables (features)
X = df[[
    'Type',
    'Days for shipment (scheduled)',
    'Category Name',
    'Customer Segment',
    'Department Name',
    'Order Region',
    'Shipping Mode',
    'order_month',
    'order_hour'
]]

# Dependent variable (target)
y = df['Late_delivery_risk']
```

In [73]:
```
# Identify categorical columns
cat_cols = X.select_dtypes(include=['object', 'category']).columns.tolist()
print('Categorical columns:', cat_cols)

# Frequency encoding (low-dimensional and robust for high-cardinality)
for col in cat_cols:
    freq = X[col].value_counts(normalize=True)
    X[f'{col}_freq'] = X[col].map(freq)

# Keep numeric columns + new encoded features, drop original string categories
X_encoded = X.drop(columns=cat_cols)
print('Shape after freq+target encoding:', X_encoded.shape)

# Use encoded features for modeling
X = X_encoded

# Train/test split after encoding
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
Categorical columns: ['Type', 'Category Name', 'Customer Segment', 'Department Na
me', 'Order Region', 'Shipping Mode']
Shape after freq+target encoding: (180519, 9)
```

In [74]:
```
y.value_counts()
```

Out[74]:
```
Late_delivery_risk
1    98977
0    81542
Name: count, dtype: int64
```

In [78]:
```
#Balancing the training data using SMOTE
print("Before balancing(train):",Counter(y_train))

smote=SMOTE(random_state=42)
X_train_bal,y_train_bal=smote.fit_resample(X_train,y_train)

print("After balancing(train):",Counter(y_train_bal))
```

```
Before balancing(train): Counter({1: 79181, 0: 65234})
After balancing(train): Counter({1: 79181, 0: 79181})
```

In [81]:
```
# Evaluate model
def evaluate_model(y_true, y_pred, model_name):
    print(f"\n--- {model_name} ---")
    print("Accuracy:", round(accuracy_score(y_true, y_pred), 2))
    print("Precision:", round(precision_score(y_true, y_pred), 2))
    print("Recall:", round(recall_score(y_true, y_pred), 2))
    print("\nClassification Report:\n", classification_report(y_true, y_pred))
```

In [87]:
```
# Train Random Forest model
print("Training Random Forest model...")

rf_model_balanced = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)
```

```
rf_model_balanced.fit(X_train_bal, y_train_bal)

print("Training completed!")

# Make predictions
y_pred_rf_balanced = rf_model_balanced.predict(X_test)

print("Predictions completed!")

# Evaluate the model
evaluate_model(
    y_test,
    y_pred_rf_balanced,
    "Random Forest Classifier"
)
```

```
Training Random Forest model...
Training completed!
Predictions completed!

--- Random Forest Classifier ---
Accuracy: 0.73
Precision: 0.77
Recall: 0.74

Classification Report:
              precision    recall  f1-score   support

           0       0.70      0.73      0.71     16308
           1       0.77      0.74      0.75     19796

    accuracy                           0.73     36104
   macro avg       0.73      0.73      0.73     36104
weighted avg       0.74      0.73      0.73     36104
```

In [ ]: